

P2447R4

std::span over an initializer list

Published Proposal, 2023-05-14

Authors:

[Arthur O'Dwyer](#)

[Federico Kircheis](#)

Audience:

LEWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Draft Revision:

5

Current Source:

github.com/Quuxplusone/draft/blob/gh-pages/d2447-std-span-over-initializer-list.bs

Current:

rawgit.com/Quuxplusone/draft/gh-pages/d2447-std-span-over-initializer-list.html

Abstract

`span<const int>` can be a lightweight drop-in replacement for `const vector<int>&` in the same way that `string_view` can replace `const string&`. While `"abc"` binds to a `string_view` function parameter, `{1, 2, 3}` fails to bind to a `span<const int>` function parameter. We show why this gap is undesirable, and propose to close it, ideally as a DR.

Table of Contents

1 Changelog

2 Background

3 Solution

3.1 Implementation experience

3.2 What about dangling?

3.3 Why not just double the braces?

3.3.1 Better performance via synergy with P2752

4 Annex C examples

4.1 Overload resolution is affected

4.2 The `initializer_list` ctor has high precedence

4.3 Implicit two-argument construction with a highly convertible `value_type`

5 Proposed wording

6 Acknowledgments

References

Informative References

§ 1. Changelog

- R4 (pre-Varna 2023):
 - Reorganize references to [\[P2752\]](#) and fix HTML goofs in proposed wording.
- R3:
 - Changed primary authorship from Federico Kircheis to Arthur O'Dwyer.
 - Removed R2's feature macro `__cpp_lib_span_init`; it didn't seem motivated.
- R2:
 - Discussed in LEWG telecon, [2022-07-26](#)

§ 2. Background

C++17 added `string_view` as a "view" over constant string data. Its main purpose is as a lightweight drop-in replacement for `const string` function parameters.

C++14 <code>string</code>	C++17 <code>string_view</code>
<pre>int take(const std::string& s) { return s[0] + s.size(); }</pre>	<pre>int take(std::string_view sv) { return s[0] + s.size(); }</pre>
<pre>std::string abc = "abc"; take(abc);</pre>	<pre>std::string abc = "abc"; take(abc);</pre>
<pre>take("abc");</pre>	<pre>take("abc");</pre>
<pre>take(std::string("abc"));</pre>	<pre>take(std::string("abc")); take(std::string_view("abc"));</pre>

C++20 added `span<const T>` as a "view" over constant contiguous data of type `T` (such as arrays and vectors). One of its main purposes (although not its only one) is as a lightweight drop-in replacement for `const vector<T>&` function parameters.

C++17 <code>vector</code>	C++20 <code>span</code>
<pre>int take(const std::vector<int>& v) { return v[0] + v.size(); }</pre>	<pre>int take(std::span<const int> v) { return v[0] + v.size(); }</pre>
<pre>std::vector<int> abc = {1,2,3}; take(abc);</pre>	<pre>std::vector<int> abc = {1,2,3}; take(abc);</pre>
<pre>take({1,2,3});</pre>	
<pre>take({}); // size=0 take({{1,2,3}}); take(std::vector{1,2,3}); take(std::initializer_list<int>{1,2,3});</pre>	<pre>take({}); // size=0 take({{1,2,3}}); take(std::vector{1,2,3}); take(std::initializer_list<int>{1,2,3}); take(std::span<const int>({1,2,3}));</pre>

This table has a conspicuous gap. The singly-braced initializer list `{1,2,3}` is implicitly convertible to `std::vector<int>`, but not to `std::span<const int>`.

§ 3. Solution

We propose simply that `std::span<const T>` should be convertible from an appropriate *braced-initializer-list*. In practice this means adding a constructor from `std::initializer_list`.

§ 3.1. Implementation experience

This proposal has been implemented in Arthur's fork of libcpp since October 2021. See "[span should have a converting constructor from initializer_list](#)" (2021-10-03) and [\[Patch\]](#).

§ 3.2. What about dangling?

`span`, like `string_view`, is specifically designed to bind to rvalues as well as lvalues. This is what lets us write useful code like:

```
int take(std::string_view s);
std::string give_string();
int x = take(give_string());

int take(std::span<const int> v);
std::vector<int> give_vector();
int x = take(give_vector());
```

Careless misuse of `string_view` and `span` outside a function parameter list can dangle:

```
std::string_view s = give_string(); // dangles
std::span<const int> v = give_vector(); // dangles
```

P2447 doesn't propose to increase the risk in this area; dangling is already likely when `span` or `string_view` is carelessly misused. We simply propose to close the ergonomic syntax gap between `span` and `string_view`.

Before	After P2447
<pre>std::string_view s = "abc"; // OK std::string_view s = "abc"s; // dangles std::span<const char> v = "abc"; // OK std::span<const char> v = "abc"s; // dangles</pre>	<pre>std::string_view s = "abc"; // OK std::string_view s = "abc"s; // dangles std::span<const char> v = "abc"; // OK std::span<const char> v = "abc"s; // dangles</pre>
<pre>std::span<const int> v = std::vector{1,2,3}; // dangles auto v = std::span<const int>({1,2,3}); // dangles std::span<const int> v = {{1,2,3}}; // dangles</pre>	<pre>std::span<const int> v = std::vector{1,2,3}; // dangle auto v = std::span<const int>({1,2,3}); // dangle std::span<const int> v = {{1,2,3}}; // dangle std::span<const int> v = {1,2,3}; // dangle</pre>

§ 3.3. Why not just double the braces?

Since we can already write

```
std::span<const int> v = {{1,2,3}}; // dangles
```

then why not call that "good enough"? Why do we need to be able to use a single set of braces?

Well, a single set of braces is good enough for `vector`, and we want `span` to be a drop-in replacement for `vector` in function parameter lists, so we need to support the syntax `vector` does. There was a period right after C++11 where some people were writing

```
std::vector<int> v = {{1,2,3}};
```

but by C++14 we had settled firmly on "one set of braces" as the preferred style (matching the preferred style for C arrays, pairs, tuples, etc.)

So I prefer to turn the question around and say: Since we can already implicitly treat `{{1,2,3}}` as a `span`, how could there be any additional harm in treating `{1,2,3}` as a `span`?

§ 3.3.1. Better performance via synergy with P2752

[P2752], sent to EWG for Varna, proposes to let a constant `initializer_list` like `{1,2,3}` refer to a backing array in static storage, rather than forcing all backing arrays onto the stack. This doesn't change anything dangling-wise: referring to the backing array of an `initializer_list` outside that `initializer_list`'s lifetime remains undefined behavior.

```
std::string_view s = "abc";           // OK, no dangling
std::span<const int> v1 = {1,2,3};    // dangles, even after P2752
```

Today, `{{1,2,3}}` converts to `span` by materializing a temporary `const int[3]` on the stack. Tomorrow, if P2447 is adopted, `{1,2,3}` will convert to `span` via an `initializer_list` that refers to a backing array on the stack. P2752 proposes to performance-optimize the latter (let the backing array occupy static storage) but not the former (keep the status quo for materialized array temporaries). In other words, the `initializer_list` constructor we propose here in P2447 is "more optimizer-friendly" than today's array-temporary constructor.

This example (Godbolt) shows how P2447 lets us benefit from P2752's optimization:

```
int perf(std::span<const int>);

int test() {
    return perf({{1,2,3}});
}
```

	<code>{{1,2,3}}</code>	<code>{1,2,3}</code>
Today	Array on stack	Ill-formed
P2752 only	Array on stack	Ill-formed
P2447 only	IL on stack	IL on stack
P2447+P2752	IL in rodata, tail-call	IL in rodata, tail-call

In each row, there's no performance difference between the single-braced or double-braced form. But the only way to reach the bottom row (tail-call, no stack usage) is to adopt both P2447 and P2752; which by a happy coincidence also permits the single-braced form.

§ 4. Annex C examples

This change will, of course, break some code (most of it pathological). We might want to add some of these examples to Annex C.

However, *any* change to overload sets (particularly the addition of new non-`explicit` constructors) can break code. For example, there was nothing wrong with C++23's adopting [P1425] "Iterator-pair constructors for `stack` and `queue`" with no change to Annex C, despite its breaking code like this:

```
void zero(queue<int>);
void zero(pair<int*,int*>);
int a[10];
```

```
void test() { zero({a, a+10}); }
```

Before: Calls `zero(pair<int, int>)`.

After P1425: Ambiguous.

To fix: Eliminate the ambiguous overloading, or cast the argument to `pair`.

We can simply agree that such examples are sufficiently unlikely in practice, and sufficiently easy to fix, that the benefits of the changed overload set outweigh the costs of running into these examples.

§ 4.1. Overload resolution is affected

```
void one(pair<int, int>);
void one(span<const int>);
void test() { one({1,2}); }
```

Before: Calls `one(pair<int, int>)`.

After P2447: Ambiguous.

To fix: Eliminate the ambiguous overloading, or replace `{1,2}` with `std::pair{1,2}`.

§ 4.2. The `initializer_list` ctor has high precedence

```
void two(span<const int, 2>);
void test() { two({{1,2}}); }
```

Before: Selects `span(const int(&)[2])`, which is non-`explicit`; success.

After P2447: Selects `span(initializer_list<int>)`, which is `explicit` for `span<const int, 2>`; failure.

To fix: Replace `{{1,2}}` with `std::array{1,2}`; or, replace `span<const int, 2>` with `span<const int>`.

§ 4.3. Implicit two-argument construction with a highly convertible `value_type`

In these two highly contrived examples, the caller deliberately constructs a `span` via its iterator-pair constructor *implicitly*, from a braced initializer of two elements, and furthermore `value_type` is implicitly convertible from the iterator type. These examples strike me as highly contrived: both conditions are unlikely, and their conjunction is unlikelier still.

```
int three(span<void* const> v) { return v.size(); }
void *a[10];
int x = three({a, 0});
```

Before: Selects `span(void**, int)`; `x` is 0.

After P2447: Selects `span(initializer_list<void*>)`; `x` is 2.

To fix: Replace `{a, 0}` with `span(a, 0)` or `span(a, a)`.

```
int four(span<const any> v) { return v.size(); }
any a[10];
int y = four({a, a+10});
```

Before: Selects `span(any*, any*)`; `y` is 10.

After P2447: Selects `span(initializer_list<any>)`; `y` is 2.

To fix: Replace `{a, a+10}` with `span(a, a+10)`.

§ 5. Proposed wording

Modify [\[span.syn\]](#) as follows:

```
#include <initializer_list>      // see [initializer.list.syn]
```

Modify [\[span.overview\]](#) as follows:

```
template<size_t N>
    constexpr span(type_identity_t<element_type> (&arr)[N]) noexcept;
template<class T, size_t N>
    constexpr span(array<T, N>& arr) noexcept;
template<class T, size_t N>
    constexpr span(const array<T, N>& arr) noexcept;
template<class R>
    constexpr explicit(extent != dynamic_extent) span(R&& r);

constexpr explicit(extent != dynamic_extent) span(std::initializer_list<value_type> il) noexcept;
constexpr span(const span& other) noexcept = default;
template<class OtherElementType, size_t OtherExtent>
    constexpr explicit(see below) span(const span<OtherElementType, OtherExtent>& s) noexcept;
```

Modify [\[span.cons\]](#) as follows:

```
constexpr explicit(extent != dynamic_extent) span(std::initializer_list<value_type> il) noexcept;

Constraints: is_const_v<element_type> is true.

Preconditions: If extent is not equal to dynamic_extent, then il.size() is equal to extent.

Effects: Initializes data with il.begin() and size with il.size().
```

§ 6. Acknowledgments

- Thanks to Federico Kircheis for writing the first drafts of this paper.

§ References

§ Informative References

[P1425]

Corentin Jabot. *Iterator-pair constructors for stack and queue*. March 2021. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p1425r4.pdf>

[P2752]

Arthur O'Dwyer. *Static storage for braced initializers*. May 2023. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2752r2.html>

[Patch]

Arthur O'Dwyer. *Implement P2447 `std::span` convertible from `std::initializer_list`*. October 2021. URL: <https://github.com/Quuxplusone/llvm-project/commit/d0d11ae5f2146d2ac76680bd1ddaf1f011f96ef4>